

AGENTS.md – Taskora

This file is loaded at the start of every AI session on this repo. It is the source of truth for what Taskora is, how it's built, and how to change it without breaking it. Read it fully before touching code.

****How to work in this repo:**** build ****one milestone at a time**** (Section 12). Say "Do milestone N from AGENTS.md", finish it, run the checks in Section 3, then stop for review and a commit. Never build several milestones in one go, and never mark a milestone done until its "Done when" list passes.

Claude Code reads `CLAUDE.md`, which contains `@AGENTS.md` to pull this file in.

****Keep it alive.**** At the end of every session, ask: ****"Is there anything from this session that should be added to AGENTS.md?"**** Add new gotchas, decisions and conventions. Remove anything that's no longer true. A stale AGENTS.md is worse than none.

1. Product

****Taskora**** is a personal task manager: a normal website you open, sign up for, log in to, and manage your own private tasks on.

****Taskora is multi-tenant.**** Many users share one deployment and one database, and every user's data is isolated from every other user's. The tenant is the individual user account. Tenant isolation is the most important rule in this codebase – read Section 4.9 before writing any query.

The product name lives in one place – `src/constants/site.ts` – change it there, never hard-code it.

1.1 Pages

Route	Access	What it does
---	---	---
`/`	public	Landing page: headline, hero illustration, clickable feature cards (signed out → sign-up modal; signed in → straight to the feature), CTAs
`/login`	guests	Bordered card: Continue with Google, or email **or** username + password; shows `error` messages from Google sign-in
`/register`	guests	Bordered card: Sign up with Google, or full name, username (live availability), email, password (live rule checklist) + **confirm password** . Success shows "Check your email" (Resend, use a different email); nobody is signed in yet
`/forgot-password`	guests	Email → "If an account uses that email, we've sent a link" (same screen whether or not the account exists)
`/reset-password?token=`	anyone with the link	Choose a new password (rule checklist); expired/used/wrong link → "This link didn't work" + "Send me a new link"
`/verify-email?token=`	anyone with the link	One button, "Verify my email" (a click, not on load – mail scanners fetch links); then "Log in"
`/dashboard`	signed in	Big weekday headline, greeting, progress, stats, quick add, Overdue, Due today, Up next, by category
`/tasks`	signed in	All tasks as a **card grid** : quick add, status tabs, category tabs, search, Filters panel, active-filter chips, clear completed
`/tasks/new`	signed in	Full task form
`/tasks/[id]/edit`	signed in	Same form, prefilled
`/trash`	signed in	Deleted tasks: Restore, Delete forever (with confirmation), Empty trash
`/profile`	signed in	Profile photo (upload / change / remove), stats, edit profile, change username, change or set password, delete account
`/robots.txt`, `/sitemap.xml`, `/opengraph-image`	public	Generated SEO files (Section 6b)
anything else	any	404 page

- Signed-out visitors hitting a signed-in page go to `/login?next=<path>` and return there after logging in or registering.
- Signed-in users hitting `/login` or `/register` go to `/dashboard`.
- `next` is only honoured if it's a same-site relative path (`safeNextPath`) – no open redirects.

1.2 Navbar

- ****Signed out:**** logo, Features (landing anchor), theme toggle, Log in, Get started.
- ****Signed in:**** logo, Dashboard, All tasks, Trash, theme toggle, ****New task**** (primary button), avatar menu (name, @username, Profile, Log out).
- ****Mobile:**** a menu button opens a panel under the header over a dimmed page.

Signed in: account card (photo, name, @username → profile), Dashboard, All tasks, Trash, ****New task****, and ****Log out**** right in the panel (no second dropdown). Signed out: Features, Log in, Get started. The active route is highlighted. It closes on an outside press (the dimmed page), Escape, the X, or choosing a link (``useDismiss``); page scroll is locked while it's open.

1.3 Tasks

A task has:

```
| Field | Rules |
|---|---|
| `title` | required, ≤ 120 chars |
| `notes` | optional, ≤ 2000 chars, multi-line (shown clamped to 2 lines, expandable) |
| `priority` | `low` \| `medium` \| `high` (default `medium`) |
| `category` | `personal` \| `work` \| `study` \| `health` \| `finance` \| `other` (default `personal`) |
| `tags` | up to 5, lowercase, ≤ 20 chars each, de-duplicated |
| `dueDate` | optional calendar day `YYYY-MM-DD` |
| `subtasks` | up to 20 checklist items `{ _id, title (≤ 120), done }` |
| `pinned` | pinned tasks always sort to the top |
| `completed` / `completedAt` | `completedAt` is set/cleared automatically |
```

Actions: create (quick add = title only, or the full form), edit, complete / uncomplete, pin / unpin, toggle a subtask inline, delete (with an in-card confirmation), clear all completed.

****Trash (soft delete).**** Deleting a task – or "Clear N completed" – moves it to the trash (``deletedAt`` is set); nothing is removed from the database. Trashed tasks are hidden from every list, filter, stat, the edit page and every update route (all return 404). On ``/trash`` a task can be ****restored**** exactly as it was, or ****deleted forever**** (in-row confirmation); ****Empty trash**** deletes them all forever (confirmation first). Only a task already in the trash can be deleted forever. No automatic purge – the trash keeps tasks until the user acts.

****Two task layouts, one behaviour:****

- ****All tasks page → cards**** (``TodoCard`` in ``TodoGrid``): 1 column on mobile, 2 on tablet, 3 on desktop). Top row: category + priority, pin button. Then a divider, checkbox + title + notes (3 lines), subtasks box (first 3, "Show all N"), due date + tags, and full-width ****Edit**** (filled) and ****Delete**** (outline) buttons pinned to the card bottom so a row of cards lines up. Delete swaps the buttons for "Move to trash? Cancel / Delete".
- ****Dashboard → compact rows**** (``TodoItem`` in ``TodoList``): same data, denser, icon buttons on hover.

List controls (all combinable, all stored in the URL so refresh/share keeps the view):

- ****Status tabs:**** All, To do, Done (with counts)
- ****Category tabs:**** always visible, one pill per category plus "All categories"; scroll sideways on mobile
- ****Search:**** title, notes and tags; case-insensitive; literal (regex characters are escaped)
- ****Filters panel**** (the Filters button): priority, due (``today``, ``overdue``, ``upcoming`` = next 7 days, ``none``) and sort. The badge counts panel filters only.
- ****Tag:**** click any ``#tag`` on a task to filter by it
- ****Active-filter chips**** (tag, priority, due, search) show above the list whenever one is on, each removable, plus "Clear all filters". Category isn't a chip – its tab already shows it.
- ****Sort:**** newest, oldest, due date (undated last), priority, title A–Z
- ****Pinned first****, always, within any sort

Dashboard stats (``GET /api/v1/todos/stats``): total, completed, active, overdue, due today, completed this week, completion %, and still-to-do counts by category and priority.

1.4 Sign in with Google

- "Continue with Google" / "Sign up with Google" on the login and register cards, shown only when ``GOOGLE_CLIENT_ID`` and ``GOOGLE_CLIENT_SECRET`` are set.
- First Google sign-in ****creates**** an account: full name from Google, username generated from the email (`"chloe.egbukwu@..."` → ``chloeegbukwu``, or ``chloeegbukwu4821`` if taken – checked through the bloom filter), no password.
- If an account with the same ****verified**** email exists, Google is ****linked**** to it. After that the account is found by Google id, even if the Google email changes.
- Google-only accounts: password login fails with the normal generic error; the profile offers ****Set a password**** (no current password needed); deleting the account is confirmed by typing the username instead of a password.
- Google accounts are ****verified from the start**** (``emailVerified: true``).
- Linking to an existing password account whose email was ****never verified**** also

- verifies it, but throws away that account's password and ends its sessions and links: whoever registered the address by password never proved they own it, so the real owner (who just signed in with Google) must not inherit their password.
- Profile shows a "Google connected" badge when linked.

1.5 Profile & account

- Shows the profile photo (or initials on a colour derived from the username), full name, @username, joined month, bio, and a stats strip.
- ****Edit:**** full name, username, email, bio (≤ 160). A ****new email is not applied straight away****: it waits in ``pendingEmail`` and a link goes to the NEW address; ``email`` changes only when that link is used. Until then the email field keeps the current address and shows "Waiting for you to confirm ..." with ****Resend****. Typing the current address back cancels the change. (Otherwise anyone could put someone else's email on their account and later take it over by signing in with Google.)
- ****Username change:**** live "Available / taken / invalid" as you type; the server validates again on save.
- ****Change password:**** new passwords need a special character (below). Needs the current password. Signs out every ****other**** device (session version bump); this device stays signed in.
- ****Profile photo:**** the camera button on the avatar picks a JPG, PNG or WebP up to 2 MB. The browser centre-crops it to a square (max 512x512, WebP or JPEG) before uploading; "Remove photo" goes back to initials. The server checks the size and the ****real type from the file's bytes**** (never the header or name). The photo is stored on the user document (``avatar``, ``select: false``) and served only to its owner from ``GET /users/me/avatar?v=<timestamp>``, so it is deleted with the account. Shown on the profile and in the navbar menu.
- ****Log out,****
- ****Delete account:**** needs the password; deletes the user and all their tasks.

1.5b Passwords, email verification, reset

- ****New passwords**** (sign-up, change, reset): 8–72 characters ****including a special character**** = anything that isn't a letter, digit or whitespace (``PASSWORD_SPECIAL`` in ``constants/todo-values.ts``, used by ``passwordField`` and by the live checklist ``PasswordRules``, so the UI can't disagree with the API). ****Login does not apply the rule****, so accounts made before it existed still sign in.
- ****Sign-up** creates the account but does not sign anyone in. ****A verification link** (24 hours, single-use) goes to the address; the person opens it, presses "Verify my email", then logs in. Logging in before that fails with 403 ``email_not_verified`` (only after the password checked out, so it never reveals which emails have accounts) and the login page offers "Resend the verification email".
- ****Accounts from before verification existed are grandfathered****: no ``emailVerified`` field means verified (``emailVerified !== false`` everywhere).
- ****Forgot password:**** email → a 30-minute, single-use reset link → new password. The answer is identical whether or not the account exists. A reset signs out every device, doesn't sign this one in, and counts as proof of the address (an unverified account that resets can log in). It also cancels an unfinished email change.
- Details, security rules and env vars: Section 4.5c.

1.6 Username rules (client and server share one schema)

- 3–20 chars; lowercase letters, numbers, underscores; must start with a letter.
- Stored lowercase; unique (MongoDB unique index is the final authority).
- Reserved words are rejected (``RESERVED_USERNAMES`` in ``src/constants/todo-values.ts``).
- ****"Is this username taken?"** uses a bloom filter first. ****"Definitely free"** is answered with zero DB queries; "probably taken" is ****always**** confirmed with a real DB lookup. Never trust the bloom filter alone for "taken". See Section 4.6. Any future feature with unique user handles must use the same approach.

2. Stack (fixed – do not switch without asking)

Layer	Choice
Framework	**Next.js 16** (App Router, Turbopack), one project for pages **and** API
Language	TypeScript (strict), pinned to <code>`~5.9`</code>
UI	React 19, Tailwind CSS v4, lucide-react icons, react-toastify
Client data	TanStack Query v5, axios
Forms	react-hook-form + zod (<code>`@hookform/resolvers`</code>)
Database	MongoDB Atlas via Mongoose 9
Auth	Encrypted httpOnly cookie sessions (iron-session <code>`sealData`</code>), bcryptjs (cost 12)
Validation	zod v4 – **one** shared schema file for client and server
Tests	Vitest (API routes called directly, in-memory fake models)

```
| Cache | MemCachier (memcached) via `memjs`; optional, off when `MEMCACHIER_SERVERS` is unset |
| Hosting | Vercel |
```

No JWT in localStorage. No separate Express server. No component library – UI primitives live in `src/components/ui`.

3. Commands & definition of done

```
```bash
npm run dev # http://localhost:3000 (needs .env.local – see .env.example)
npm run typecheck # tsc --noEmit
npm run lint # eslint . (flat config, eslint-config-next)
npm test # vitest – no database needed
npm run build # next build (no secrets needed at build time)
```
```

****A change is done only when all four pass:**** `typecheck`, `lint`, `test`, `build`. For UI changes, also look at the page at ~400px wide and on desktop, in light ****and**** dark mode. Never hand over unverified code.

4. Architecture

4.1 Folder structure

```
```
src/
 app/
 layout.tsx # Next.js routes ONLY – thin files
 providers.tsx # <html>, fonts, pre-paint theme script, <Providers>
 globals.css # QueryClient, ThemeProvider, ToastContainer
 not-found.tsx, error.tsx # design tokens + signature CSS (ink strike, checkmark, toasts)
 (marketing)/ # public pages: layout (navbar+footer), page.tsx = landing
 (auth)/ # layout wraps <GuestOnly>; login/, register/, forgot-password/
 (account)/ # pages opened from an emailed link (no GuestOnly): reset-password/,
 verify-email/ # layout wraps <RequireAuth>; dashboard/, tasks/, tasks/new/,
 (app)/
 tasks/[id]/edit/, profile/
 robots.ts, sitemap.ts, opengraph-image.tsx # SEO files (Section 6b)
 api/health/route.ts
 api/v1/**/route.ts # one-line handlers: export const GET = route(controllerFn)
 proxy.ts # cookie-presence redirects (Next 16 name for middleware)
 server/ # SERVER-ONLY code (every file imports 'server-only')
 config/env.ts, db.ts # env validation (lazy), cached Mongoose connection
 models/ # *.model.ts – Mongoose schemas
 controllers/ # *.controller.ts – the actual request logic
 services/ # *.service.ts – username bloom filter, email.service (Brevo), email-
 templates, account-email, cache.service (memjs, Eventra's format), user-cache.service (per-user,
 encrypted, versioned)
 lib/ # http.ts (envelope, route wrapper, parsing), session.ts, auth.ts, rate-
 limit.ts, helpers.ts
 test/ # fake-model.ts, client.ts (TestClient), api.test.ts, empty.ts
 views/ # one client component per page: *-view.tsx
 components/
 ui/ # primitives: button, input, select, field (+Textarea), avatar,
 password-input
 layout/ # navbar, mobile-menu, nav-link, user-menu, footer, page-wrapper
 profile/ # avatar-upload (camera button, crop, upload)
 todos/ # todo-card + todo-grid (All tasks), todo-item + todo-list (dashboard),
 todo-toolbar, todo-form, quick-add, tags-input, subtasks-editor, todo-empty-state
 marketing/ # hero-illustration (SVG)
 auth/ # username-status, password-rules, check-email, auth-shell (the centred
 card), google-button
 guards/ # require-auth (also provides useCurrentUser), guest-only
 skeletons/, shared/
 hooks/ # auth/, profile/, todos/, shared/ (use-dismiss: outside press + Escape)
 – TanStack Query hooks
 api/ # client fetchers: client.ts (axios + ApiError), auth.ts, users.ts,
 todos.ts
 lib/
 validation.ts # ★ THE shared zod schemas (API bodies + field rules)
 schema.ts # client form schemas built from validation.ts
```
```

```

  utils.ts, form-errors.ts
  crop-image.ts           # browser-side square crop for profile photos
  site-url.ts             # the public origin (APP_URL) for canonical links, sitemap, share tags
constants/
  site.ts                 # product name, tagline, description (SEO), author
  todo-values.ts          # enums + LIMITS + reserved usernames (shared, no React/Mongoose)
  todo.ts                 # UI metadata: labels, icons, colours, sort/due options
context/theme-context.tsx
types/                    # user.ts, todo.ts – mirror the API's public shapes
``

```

Naming: files are kebab-case; server files carry a layer suffix
 (`todo.controller.ts`, `user.model.ts`, `username-bloom.service.ts`). Imports
 always use the `@/` alias.

4.2 The page pattern

`app/\*\*/page.tsx` is a small **server** component that exports ``metadata`` and
 renders one client view from ``src/views/``. Pages whose view calls
``useSearchParams`` (tasks, login, register) wrap it in ``<Suspense>`` – Next
 requires it. Don't put hooks or ``use client`` in ``page.tsx``.

4.3 The API pattern

```

``ts
// src/app/api/v1/todos/route.ts
export const GET = route(listTodos)
export const POST = route(createTodo)
``

```

- ``route()`` (in ``server/lib/http.ts``) connects to MongoDB, runs the controller,
 and turns anything thrown into the standard envelope. Controllers never
 write `try/catch` – they **throw** ``HttpError`` / ``fieldError``.
- Dynamic params are a **Promise** in Next 16: ``route<{ id: string }>(async (req, { params }) =>
 getTodo(req, (await params).id))``.
- Parse input only with ``parseBody(req, schema)`` / ``parseQuery(req, schema)`` –
 bad JSON → 400, invalid → 400 with ``details``.
- Update bodies are `strict()` so clients can't set ``userId``, ``password``, etc.

Envelope (every response):

```

``json
{ "success": true, "message": "Task added", "body": { } }
{ "success": false, "message": "Validation failed", "details": [{ "path": "title", "message": "Give the  

task a name" }] }
``

```

Forms map ``details`` onto fields with ``applyServerErrors`` (``src/lib/form-errors.ts``).
 Errors may also carry a machine-readable ``code`` (``email_not_verified``, ``invalid_token``,
``email_taken``); the client reads it with ``errorCode()`` and ``ApiError.code``.

4.4 API reference (base ``/api/v1``)

| Method & path | Auth | Body / query | Returns |
|---|------|--|--|
| <code>`POST /auth/register`</code> | – | <code>`{ fullName, username, email, password }`</code> | 201 <code>`{ email, verificationSent }`</code> ; no session cookie ; emails a verification link |
| <code>`POST /auth/verify-email`</code> | – | <code>`{ token }`</code> <code>`{ email }`</code> | 400 <code>`invalid_token`</code> if used/expired/wrong |
| <code>`POST /auth/resend-verification`</code> | – | <code>`{ identifier }`</code> (email or username) | same message always; sends only if the account needs verifying (or has a pending email) |
| <code>`POST /auth/forgot-password`</code> | – | <code>`{ email }`</code> | same message always; emails a reset link only if the account exists |
| <code>`POST /auth/reset-password`</code> | – | <code>`{ token, newPassword }`</code> | success message; 400 <code>`invalid_token`</code> ; no session cookie |
| <code>`POST /auth/login`</code> | – | <code>`{ identifier, password }`</code> (email or username) | user + session cookie; 403 <code>`email_not_verified`</code> |
| <code>`POST /auth/logout`</code> | – | – | clears cookie |
| <code>`GET /auth/google`</code> | – | <code>?next=`</code>                                             | 302 to Google (sets the short-lived <code>`taskora_oauth`</code> cookie) |
| <code>`GET /auth/google/callback`</code> | – | <code>?code&state`</code> from Google                        | 302 into the app with a session, or to <code>`/login?error=...`</code> |
| <code>`GET /auth/me`</code> | ✓ | – | user (401 + cookie cleared if invalid) |
| <code>`GET /users/username-available`</code> | – | <code>?username=`</code>   <code>`{ available, reason? }`</code> | |
| <code>`GET /users/me/avatar`</code> | ✓ | – | the image bytes (404 if none) |
| <code>`PUT /users/me/avatar`</code> | ✓ | raw image bytes (jpg/png/webp, ≤ 2 MB) | user (with <code>`avatarUrl`</code>) |
| <code>`DELETE /users/me/avatar`</code> | ✓ | – | user (<code>`avatarUrl: null`</code>) |
| <code>`PATCH /users/me`</code> | ✓ | any of <code>`{ fullName, username, email, bio }`</code> | user (a new <code>`email`</code> becomes <code>`pendingEmail`</code> and a link is sent there) |

```
| `PATCH /users/me/password` | ✓ | `{ currentPassword?, newPassword }` (current required only if the
account has a password) | fresh cookie; message "Password changed" or "Password set" |
| `DELETE /users/me` | ✓ | `{ password }`, or `{ confirmPassword }` for Google-only accounts | clears
cookie |
`GET /todos`	✓	`?status&priority&category&tag&due&search&sort&today`	`{ todos, stats }`
`GET /todos/stats`	✓	`?today`	stats
`POST /todos`	✓	task fields (only `title` required)	task (201)
`GET /todos/[id]`	✓	-	task
`PATCH /todos/[id]`	✓	any task fields + `completed`	task
`DELETE /todos/[id]`	✓	-	`{ _id }` - moves it to the trash
`PATCH /todos/[id]/subtasks/[subtaskId]`	✓	`{ done }`	task
`DELETE /todos/completed`	✓	-	`{ deletedCount }` - moves completed tasks to the trash
`GET /todos/trash`	✓	-	`{ todos }` trashed, most recently deleted first
`DELETE /todos/trash`	✓	-	`{ deletedCount }` - empties the trash (permanent)
`POST /todos/[id]/restore`	✓	-	task (404 unless it's in the trash)
`DELETE /todos/[id]/permanent`	✓	-	`{ _id }` (404 unless it's in the trash)
`GET /api/health`	-	`?check=email` (optional)	`{ status, database, email: 'configured' \ 'not
configured' }`; with `?check=email` also `emailCheck` (see Section 4.5c) |
```

`today` is the viewer's own `YYYY-MM-DD` (the client always sends it). Due filters and stats are calendar-day based and the server can't know the user's timezone.

4.5 Auth & sessions

- The session is an **encrypted, signed cookie** ``taskora_session`` (``sealData``, 30 days, `httpOnly``, ``sameSite: 'lax'``, ``secure`` in production). It holds only `{ uid, v }` - user id and session version.
- ``requireUser(req)`` (server/lib/auth.ts) unseals the cookie, loads the user, and checks ``sessionVersion``. Any failure → 401 **and the cookie is cleared**.
- **Password change** bumps ``user.sessionVersion`` → every other device's cookie is now invalid; the current device gets a fresh cookie.
- **Login** compares against a dummy hash when the user doesn't exist, so timing can't reveal which emails are registered, and uses one error message for "no such user" and "wrong password".
- ``proxy.ts`` only checks whether the cookie **exists** (fast redirects). Real authorization happens in every API route. Because the API clears invalid cookies on 401, a stale cookie can't cause a redirect loop.
- Client side: ``useMe()`` resolves to ``User | null`` (401 = null, not an error). ``<RequireAuth>`` renders the signed-in shell and provides ``useCurrentUser()`` via context; it keeps the last known user for the one frame between logout and navigation so nothing crashes.
- On login/register the client removes all cached non-auth queries (another person may have used the browser); on logout/delete it clears the whole cache.

4.5b Google OAuth (`server/controllers/google.controller.ts`)

- Authorization-code flow with **PKCE** and a **state** check, no auth library. ``state``, the PKCE verifier and ``next`` travel in an encrypted cookie (``taskora_oauth``, 10 minutes, ``sameSite: 'lax'`` - Google returns with a top-level GET, so ``strict`` would drop it).
- The code is exchanged server-to-server; the profile comes from Google's `userinfo` endpoint over TLS. **Only ``email_verified: true`` is trusted** - otherwise someone could claim another person's email and hijack the link.
- Every failure redirects to ``/login?error=<code>`` (``google_failed``, ``google_cancelled``, ``google_unverified``, ``google_unavailable``, ``too_many_attempts``); ``login-view.tsx`` maps codes to messages.
- User fields: ``googleId`` (sparse unique, ``select: false``), ``hasPassword``. ``toPublicUser`` exposes ``hasPassword`` and ``googleLinked``, never ``googleId``. ``hasPassword`` is derived from the password itself whenever the caller selected ``+password`` (``requireUser``, login, profile and avatar updates do), so accounts created before the flag existed still get "Change password".
- Redirect URI: ``${APP_URL || request origin}/api/v1/auth/google/callback``. It must be listed **exactly** in Google Cloud Console (see Section 10).

4.5c Email verification and password reset

Files: ``controllers/email-auth.controller.ts`` (verify, resend, forgot, reset), ``services/account-email.service.ts`` (make a token, store its hash, send), ``services/email.service.ts`` (Brevo's transactional API over ``fetch``, no SDK), ``services/email-templates.ts``, ``lib/tokens.ts``. Register/login/profile/Google changes are in their own controllers.

- **Tokens**: 256 random bits (``base64url``) in the email; only the **SHA-256 hash** and an expiry are stored on the user (``verifyTokenHash/Expires``, ``resetTokenHash/Expires``, all ``select: false``). One outstanding token per purpose: a new link replaces the old.

Verify links last 24 h, reset links 30 min. Used tokens are cleared, and the update is conditional on the hash (``updateOne({ _id, resetTokenHash: hash }, ...)` + `modifiedCount``), so a link can't be used twice even by two simultaneous requests.

- ****POST, not GET,**** for verify: mail scanners and link previews fetch URLs; a GET would burn the token. The page waits for a click.
- ****Validate before consuming:**** the reset password is checked by zod *before* the token is looked up, so a weak password doesn't use up the link.
- ****Generic answers:**** forgot / resend answer identically whether or not the account exists (send failures are logged, never returned). Rate limits: per IP and per address (Section 4.8).
- ****Links are built from `siteUrl()` (`APP\_URL`), never from the request's Host header,**** which an attacker controls (a poisoned Host would send the victim a link to the attacker's site).
- ****Reset**** bumps ``sessionVersion`` (all devices signed out), sets ``emailVerified``, and clears any pending email change and its link. ****Changing your password**** clears an unused reset link.
- ****`pendingEmail`:**** see Section 1.5. The verify endpoint applies it (and re-checks the address isn't taken by then: 409 ``email_taken``).
- ****Email transport:**** Brevo, ``POST https://api.brevo.com/v3/smtp/email`` with the ``api-key`` header. Needs ****both**** ``BREVO_API_KEY`` (an API key, "xkeysib-...", from SMTP & API → API keys; not the SMTP password) and ``EMAIL_FROM`` (``Taskora <me@example.com>`` or a bare address, parsed by ``parseSender``). Development without them prints the email (link included) to the terminal. ****Production without them answers 503 before creating anything**** (``assertEmailReady``) rather than pretend to send. ``/api/health`` shows ``email: configured``. Swapping providers means rewriting ``sendEmail()`` only.
- ****Brevo sender:**** ``EMAIL_FROM`` must be a sender verified in Brevo (Senders, domains & dedicated IPs); any other address is rejected (400) and the reason is in the server log (``Brevo responded 400: ...``, never the API key). Free plan: 300 emails/day. ****A Yahoo/Gmail-style address as the sender is unreliable****: those providers publish strict DMARC policies, so mail "from" them that is sent through Brevo can land in spam or be rejected. Verify a domain in Brevo (add its DNS records) and send from an address on it as soon as there is one. A provider failure at sign-up doesn't lose the account: the response says ``verificationSent: false`` and the page offers Resend.

****Diagnosing "I'm not getting emails".**** Open ``/api/health?check=email``. It asks Brevo (read-only, nothing is sent, 5 calls per 15 min per IP) and answers ``apiKey`` (valid / rejected), ``sender`` (is ``EMAIL_FROM`` an active verified sender), ``transactional`` (has Brevo switched sending on) and a plain-language ``hint``. Two things it can't see: a message Brevo accepted but a mailbox filtered (check spam and Brevo → Transactional → Logs), and an address with no account (forgot-password answers 200 and sends nothing on purpose). In the Vercel log a request that tried to send shows an outgoing call to ``api.brevo.com`` under ****External APIs****; ****"No outgoing requests" means nothing was sent****. Brevo's usual blockers: the account's *Authorised IPs* setting (Vercel's IPs change, so blocking must be off), an unverified sender, or transactional sending not yet activated.

4.6 Username bloom filter (``server/services/username-bloom.service.ts``)

- Sized for 100k usernames at 1% false positives; two FNV-1a hashes combined by double hashing.
- Built lazily from the DB on first use per server instance (self-heals after a cold start); new/renamed usernames are added immediately.
- Renamed-away or deleted usernames stay in the filter – harmless, because "probably taken" is always confirmed with ``User.exists(...)``.
- ****Hash values must stay unsigned**** (``>>> 0``). JS ``^`` returns a signed int; a negative bit index is silently ignored by ``Uint8Array``, which produced false "available" answers before the fix. The test "never gives a false negative" guards this.

4.7 Database

- ``connectDB()`` caches one Mongoose connection on ``globalThis`` (survives dev hot reload and warm serverless invocations). ``bufferCommands: false``.
- Models use ``mongoose.models.X || mongoose.model(...)`` to survive hot reload.
- Every task query filters by ``userId`` – see ****Section 4.9 Multi-tenancy**** for the full rules.
- ``userId``, ``password``, ``sessionVersion``, the email token fields and ``__v`` are never sent to the client.
- Index: ``{ userId: 1, completed: 1, createdAt: -1 }`` and ``{ userId: 1, deletedAt: -1 }`` (trash); unique indexes on ``username`` and ``email``.
- ****Soft delete:**** every live-task query adds ``LIVE = { deletedAt: null }`` (in ``in`todo.controller.ts``); trash queries use ``{ deletedAt: { $ne: null } }``. ``deletedAt: null`` also matches old documents with no field. Any new task query must include one of the two, or trashed tasks leak back into the UI.
- A Mongo duplicate-key error (11000) becomes a 409 with a field detail – that covers two people registering the same username at the same instant.

4.7b Read cache (MemCachier)

Optional (same `MEMCACHIER\_SERVERS` / `\_USERNAME` / `\_PASSWORD` variables as Eventra). `GET /todos`, `GET /todos/stats` and `GET /todos/trash` are cached for 60 s; responses carry `x-cache: HIT | MISS` (absent when the cache is off or paused). Without `MEMCACHIER\_SERVERS` nothing changes.

- **Eventra's format, adapted.** `services/cache.service.ts` is Eventra's: lazy `memjs` client with MemCachier
SASL, `timeout: 1, retries: 1, failover: false`, `getCache/setCache/deleteCache/flushCache` that **never throw**,
key prefix `tk:v1` (Eventra's is `jc:v1`), 60 s default TTL. Eventra caches public pages by URL through
middleware; that would leak here, so Taskora's private data goes through `services/user-cache.service.ts`
(`cachedForUser`, `invalidateUserCache`, `withCacheHeader`) called from the controllers, never through the raw
primitives and never keyed by URL.
- **Tenant isolation.** Keys are `tk:v1:u:<userId>:v<dataVersion>:<scope>:<sha256 of the query>`; `userId` and
`dataVersion` come from `requireUser`. Authentication is never cached: `requireUser` still hits MongoDB on
every request.
- **Encrypted.** Values are sealed with AES-256-GCM (key derived from `SESSION\_SECRET` with HKDF) before they
go to the third-party server, with the cache key as authenticated data: MemCachier holds only ciphertext, a
value moved under another key or altered is a miss. Changing `SESSION\_SECRET` just empties the cache.
- **Invalidation by version, kept in MongoDB.** Every write to a user's tasks (`create`, `update`, subtask toggle,
`delete`, clear completed, `restore`, delete forever, empty trash, delete account) ends with
`await invalidateUserCache(userId)`, which does `\$inc: { dataVersion: 1 }` on the user. Keys embed the version, so
older copies become unreachable at once. The counter lives in the database (not the cache) so it survives
the cache being down, evicting or flushed during a write, which is exactly when a cache-side counter would leave
stale data behind. It is skipped when MemCachier isn't configured.
- **A new task-write path must call `invalidateUserCache`;** a new cached read must use `cachedForUser`.
- **Failure is invisible.** A miss, an error or a full cache means reading MongoDB. After any cache failure a
circuit breaker skips the cache for 30 s per server instance, so a hanging MemCachier costs one slow request
(about 1 s), not every request. Sets are awaited (a serverless function can be frozen once it responds).
- **Not cached:** `GET /todos/[id]` and everything auth-related.
- **Tested** with an in-memory `memjs` stand-in (hit/miss, every write invalidates, tenants separate, ciphertext
only, tamper and swap ignored, outage) and, by hand, against a real `memcached` with the real client and a real
browser (`memcached -p 11311 -d`, `MEMCACHIER\_SERVERS=127.0.0.1:11311`). MemCachier's SASL login can't be tested
without an account; the options are the ones Eventra runs in production.

4.8 Rate limits (`server/lib/rate-limit.ts`)

| Action | Limit |
|---|---|
| login / register / change password / delete account | 10 per 15 min per IP |
| create task | 30 per minute per user |
| upload / remove profile photo | 20 per 15 min per user |
| send verification / reset email | 5 per 15 min per IP and 3 per 15 min per address |
| try a verify / reset link | 30 per 15 min per IP |
| username availability check | 120 per minute per IP |

In-memory per server instance → best-effort on Vercel. `DISABLE\_RATE\_LIMIT=1` turns it off (local e2e harness only). Swap for Upstash Redis if it ever matters.

4.9 Multi-tenancy (tenant isolation)

Model: pooled / row-level multi-tenancy. All tenants share one MongoDB database and the same collections. Every tenant-owned document carries the tenant key, and every query filters by it.

| | |
|--------------------------------|----------|
| One user account | (`User`) |
| No organisations or teams yet. | |

```
| **Tenant key** | `userId` (ObjectId) on every tenant-owned document |
| **Tenant-owned data** | `todos` (including their subtasks, tags, notes and trashed ones), the profile
photo (stored on the user) |
| **Global (shared) data** | `users` itself; the **username** and **email** namespaces are unique across
all tenants, which is why the username bloom filter is one global filter |
| **Where the tenant comes from** | Only the encrypted session cookie, via `requireUser(req)` |
```

****The rules – every one is mandatory:****

- **The tenant id comes from the session, never from the request.****
`const { userId } = await requireUser(req)` is the only source. Never read a user or tenant id from the body, query string, route params or headers. Update bodies are `.strict()` so a client can't smuggle in `userId`.
- **Every query on tenant-owned data includes `userId`: find, findOne, count, update, delete and aggregate. Fetch by id with**
`{ \_id: id, userId },` never `findById(id)` on a tenant-owned model.
- **Another tenant's data returns 404, never 403.**** A 403 would confirm that the id exists. Treat "belongs to someone else" exactly like "doesn't exist".
- **Writes stamp the tenant from the session:**** `Todo.create({ ...input, userId })`, and `userId` is never updatable.
- **Responses never expose the tenant key:**** `userId` is stripped (`PUBLIC\_FIELDS`) and the client never needs it.
- **Stats and aggregates are per tenant:**** `loadStats(userId, ...)` only ever counts that user's documents. Never compute anything across tenants in a user-facing endpoint.
- **Indexes lead with the tenant key**** (`{ userId: 1, completed: 1, createdAt: -1 }`), so tenant-scoped queries stay fast as the number of users grows.
- **Deleting a tenant deletes all their data:**** `deleteAccount` removes every `todos` document for that `userId`, then the user. Any new tenant-owned collection must be added to that cascade.
- **Client caches are per tenant.**** On login and register the TanStack Query cache drops everything except auth; on logout and account deletion it's cleared completely. A second person on the same browser must never see the first person's cached tasks.
- **Rate limits are per tenant where there is one**** (task creation is keyed by `userId`) and per IP before sign-in.
- **Anything cached is per tenant too**** (Section 4.7b): key starts with the session's `userId`, value is encrypted with the key bound in, and every write invalidates. Never cache tenant data by URL.
- **Isolation is tested.**** `api.test.ts` has a second user try to read, update and delete the first user's task (all 404) and list tasks (empty); the browser test checks a second user sees an empty list. Any new tenant-owned endpoint needs the same "another user gets 404" test.

****Adding a new tenant-owned model**** (e.g. projects, comments, attachments):
add `userId: { type: ObjectId, ref: 'User', required: true, index: true }`,
lead its compound indexes with `userId`, scope every controller query with it, strip it from responses, add the collection to the account-deletion cascade, and add a cross-tenant 404 test.

****If teams/workspaces are added later**** (roadmap), the tenant changes from "user" to "workspace": documents gain `workspaceId`, `requireUser` becomes "require membership of this workspace", and every rule above applies to `workspaceId` instead. Decide and spec that in this file before building it.

5. Client rules

- **Server state = TanStack Query only.**** Never copy query data into `useState`.
- Query keys come from the key factories (`todoKeys`, `authKeys`) – never hand-write arrays.
- **Mutations are optimistic**** (`useOptimisticLists` in `hooks/todos/use-todo-actions.ts`): patch every cached list, roll back on error, and ****always invalidate** `todoKeys.all` on settle – the server decides which filtered list a task belongs to, and stats/detail caches must refresh too.
- **Filters live in the URL**** (`useTodoFilters`): defaults are omitted, unknown values fall back to defaults, updates use `router.replace(..., { scroll: false })` and read the live `window.location.search` so rapid changes don't clobber each other.
- **Forms:**** react-hook-form + zod. Field rules come from `lib/validation.ts` (shared with the API). Send only changed fields on edit.
- **Dates:**** `dueDate` is a `YYYY-MM-DD` string. Parse with `date-fns/parseISO` (local midnight). ****Never**** `new Date('YYYY-MM-DD')` – that's UTC and shifts the day.
- Username availability: `useUsernameAvailability(value, currentUsername?)` – debounced 350ms, format errors caught locally with no request.
- Every `use client` boundary is deliberate: `page.tsx` files stay server components.

6. Design system (don't drift)

The feel: a calm planner. Cool paper-blue in light mode, deep navy in dark. The day is the hero. Completing a task feels like ticking it off with a pen.

- **Tokens** (`globals.css`, `:root` + `.dark`): `background`, `surface`, `foreground`, `muted-foreground`, `border`, `primary`, `primary-soft`, `done`, `destructive`, `warning`. Use token classes (`bg-surface`, `text-done`) – **never raw hex** in components.
 - Light: background `#F3F6FC`, surface `#FFFFFF`, ink `#16233F`, primary `#2F5BEA`, done `#16936B`
 - Dark: background `#0E1524`, surface `#162036`, ink `#E7EDF8`, primary `#7593FF`, done `#3CC495`
- **Type**: Bricolage Grotesque (`font-display`) for headlines, headings, stats and empty states; Geist (`font-sans`) for everything else.
- **Signature moments** – keep them, don't add more:
 1. The dashboard headline is **today's weekday**, huge (`clamp(2.75rem, 11vw, 4.5rem)`), date underneath.
 2. Completing a task draws an **ink strike-through** (`.ink-strike`) and the check **draws itself** (`.check-path`).
- **Hero illustration** (`components/marketing/hero-illustration.tsx`): an original SVG of the app – window, category tabs, two task cards with Edit / Delete buttons, a "done" badge and a progress ring. Drawn only with token classes (`fill-surface`, `stroke-border`, `fill-primary`...) so it follows dark mode with no image file. It contains **no dates or live data** – the landing page is prerendered, so anything time-based would mismatch on hydration.
- **Auth pages**: one centred card (`rounded-xl border bg-surface`, max-w-md) with the logo directly above it – never at the far top of the page. Heading centred with no tagline under it on login/register (the form starts straight away); the form first, then an "or" divider, then the Google button.
- **Modals**: `components/ui/modal.tsx` wraps the native `` (`showModal()`): focus trap, Esc, backdrop click and `::backdrop` blur for free. Use it for every modal; don't hand-roll overlays.
- **Cards**: `rounded-lg border bg-surface`, pinned cards get a `border-primary/40` outline, soft primary shadow on hover. Pills (category tabs) are `rounded-full`; the selected pill is filled `bg-primary`.
- **Anything that pops open** must close on an outside press and Escape, not only on its own Cancel/X: use `useDismiss(ref, active, onClose, ignoreRef)` (`hooks/shared/use-dismiss.ts`) for menus, the mobile panel and inline confirmations (task-card delete, trash delete-forever, empty trash, delete account). Pass the opening button as `ignoreRef` so its own click doesn't fight the dismissal. `` already closes on a backdrop click.
- **Toasts**: `ToastContainer` sits **top-right**, just below the navbar (`providers.tsx` + the `.Toastify\_\_toast-container--top-right` rule in `globals.css`), so they never cover New task or the avatar menu; full width at the top on phones. Every user-visible change gets one, reusing the button's verb: task added, updated, completed / marked as to do, pinned / unpinned, moved to trash, restored, deleted forever; profile updated, password changed / set, photo updated / removed; logged in / out, account deleted. Failures use `toast.error(error.message)` unless the error is shown on a form field. Subtask ticks are intentionally silent (too frequent).
- Motion otherwise minimal; `prefers-reduced-motion` disables all transitions globally.
- **Layout**: centred column (`PageWrapper`: narrow 2xl for forms/profile, default 4xl for the dashboard, wide 6xl for landing and the task grid), left-aligned text.
- **Copy**: sentence case, plain verbs. Buttons say what they do ("Add task", "Save changes", "Delete account and all tasks"). Toasts reuse the button's verb ("Task added"). Errors say what happened and what to do. Empty states invite an action. No ALL-CAPS labels. No meta strings joined with middle dots.
- **Theme**: the class `dark` on ``, set **before first paint** by an inline script in `app/layout.tsx` (key `taskora-theme`, must match `theme-context.tsx`). `` is required for that. The theme toggle renders both icons and lets CSS pick, so server HTML never mismatches.
- **Accessibility**: every icon button has an `aria-label`; the task checkbox is `role="checkbox" + aria-checked`; tabs use `role="tab" + aria-selected`; the pin control is `role="switch"`; visible focus rings everywhere; form errors are linked to fields; progress bars use `role="progressbar"`.

6b. SEO

- **Public origin**: `siteUrl()` (`lib/site-url.ts`) = `APP\_URL`, else Vercel's production URL, else `http://localhost:3000`. It feeds `metadataBase`, canonical links, the sitemap, `robots.txt` and the JSON-LD. It is read **at build time**, so set `APP\_URL` on Vercel and redeploy after changing the domain.
- **Root layout** (`app/layout.tsx`): title template `%s | Taskora`, `SITE.description`,

keywords, canonical `/`, Open Graph and Twitter (`summary\_large\_image`). The share picture is `app/opengraph-image.tsx` (1200×630, drawn from token colours; Twitter/X reuse it). Copy lives in `constants/site.ts`; never hard-code it.

- ****Landing page**** has its own absolute title and description and `WebApplication` JSON-LD (`<` is escaped in the inline script). Login and register have their own descriptions and canonicals (`?next=` / `?error=` variants all point at one page).
- ****Private pages are never indexed:**** the `(app)` layout exports `robots: { index: false, follow: false }`; `/api/\*` sends `X-Robots-Tag: noindex, nofollow` (`next.config.ts`); `robots.txt` disallows `/api/`, `/dashboard`, `/tasks`, `/trash`, `/profile`, `/forgot-password`, `/reset-password`, `/verify-email` (the last two also send `referrer: no-referrer`, since their URL holds a secret token); the 404 page is noindex automatically.
- ****Sitemap**** lists only public pages (`/`, `/register`, `/login`). A new public page must be added to `app/sitemap.ts`; a new signed-in page goes under `(app)` (noindex for free) and into the `robots.txt` disallow list.
- Signed-in pages use the client-side `RequireAuth`, so crawlers only ever see the public pages; SEO effort belongs on the landing, login and register pages.

7. Security checklist (review on every backend change)

- [] Tenant isolation rules in Section 4.9 hold (tenant id from the session only, every tenant-owned query scoped by `userId`, 404 not 403, cascade delete, cross-tenant test).
- [] Every protected handler calls `requireUser(req)` first.
- [] Every task query includes `userId`.
- [] Input validated with the shared zod schemas; update bodies are `.strict()`.
- [] No secrets or internal fields in responses (`toPublicUser`, `PUBLIC\_FIELDS`).
- [] Search input passes through `escapeRegExp`.
- [] Task queries include `LIVE` or `TRASHED` (soft delete, Section 4.7).
- [] Cached tenant data goes through `cachedForUser` (userId + dataVersion in the key, encrypted); every new write path calls `invalidateUserCache`.
- [] Uploaded files are checked on the server by their bytes, never by the Content-Type header or file name.
- [] Auth-sensitive endpoints are rate limited.
- [] Redirect targets pass `safeNextPath`.
- [] Emailed links use `siteUrl()`, never the Host header; tokens are stored hashed, single-use, expiring; endpoints that email answer the same for known and unknown accounts.
- [] New popovers / inline confirmations use `useDismiss`.
- [] New signed-in pages are under `(app)` (noindex) and disallowed in `robots.ts`; new public pages are in `sitemap.ts`.
- [] Security headers set in `next.config.ts` (nosniff, frame DENY, referrer, permissions).

8. Testing

- `npm test` runs `src/server/test/api.test.ts` (84 tests): bloom filter correctness, auth, sessions, forged cookies, rate limiting, profile, username changes, password change signing out other devices, account deletion, Google sign-in (create, username clash, link by email, find by id, bad state, unverified email, failed token exchange, not configured, Google-only password and deletion rules), profile photo (upload, replace, remove, type sniffing, size, owner-only), trash (soft delete hides the task everywhere, restore, delete forever only from the trash, empty, per-user privacy), email verification (login blocked until verified, single-use / expiring / wrong tokens, resend replaces the old link, legacy accounts, provider outage, no provider in dev vs prod, APP_URL not Host, HTML escaping, Google verifying and discarding a stranger's password), forgot / reset (generic answers, all devices signed out, weak password doesn't burn the link, wrong-kind tokens, per-address limit), pending email changes, the special-character rule, logout not touching the database, the `/api/health?check=email` diagnostics (healthy, blocked IP, bad key, unverified sender, transactional off, unconfigured, unreachable, rate limit), the memcached read cache (miss/hit, every write invalidates, per-user separation, ciphertext only, tamper/swap ignored, a write during an outage never leaves stale data, outage and circuit breaker), task privacy, filters, sorting, stats, and validation.
- ****Test helpers:**** every `TestClient` gets its own IP (per-IP rate limits don't couple people); `signIn()` registers, marks the address verified, then logs in; `beforeEach` stubs Brevo's HTTP API into `outbox` (`lastEmail`, `tokenOf`, `sentTo`). New code that sends mail is tested through that outbox.
- The tests import the ****real route handlers**** from `src/app/api/\*\*/route.ts` and call them with `NextRequest`s through `TestClient`, which keeps cookies like a browser (one `TestClient` = one person, all cookies in a jar; `upload()` sends raw bytes). Google is tested by stubbing `fetch` for the token and userinfo endpoints.

- ``vi.mock`` swaps the models for ``fake-model.ts`` (in-memory Mongoose look-alike) and ``connectDB`` for a no-op. ****If a controller uses a new Mongoose method or query operator, add it to ``fake-model.ts`` or the tests will fail.**
- ``server-only`` is aliased to an empty module in ``vitest.config.mts``.
- End-to-end checks are done by building a copy of the app with the fake models, running ``next start``, and driving it with Playwright. When doing this, copy ``node_modules`` (Turbopack rejects a symlinked ``node_modules``), and set ``DISABLE_RATE_LIMIT=1``. To follow email links in the browser, patch the copy's ``email.service.ts`` to append each message to a file and read the link from it.

9. Gotchas already hit (don't repeat them)

****Next.js 16****

- Middleware is now ****`proxy.ts`** exporting ``proxy()``. ``middleware.ts`` is deprecated.
- Route handler ``params`` and page ``params`` are ****Promises**** – ``await`` them.
- ``next lint`` is gone; ESLint runs as ``eslint .`` with a flat config importing ``eslint-config-next/core-web-vitals`` and ``/typescript``.
- ``useSearchParams`` in a client component needs a ``<Suspense>`` boundary in the page.
- Next renders a hidden ``role="alert"`` route announcer – in tests, don't select "the alert"; select your message text.
- Don't use ``next/font/google`` if the build machine can't reach Google; fonts load via ``<link>`` in ``app/layout.tsx`` (and ``@next/next/no-page-custom-font`` is disabled on that line on purpose).
- Use ``<Link>`` for internal links, including ``/#features`` – plain ``<a>`` fails lint.

****React / UI****

- ****Theme = ``useSyncExternalStore`` over the ``dark`` class on ``<html>`` with a ``light`` server snapshot. Reading ``localStorage/matchMedia`` in a ``useState`` initializer made the toggle's ``aria-label`` differ between server and client (hydration mismatch, only visible in ``npm run dev``). Check theme changes in dev mode with a saved dark preference.**
- The navbar renders on public pages too (landing, 404). Anything in it must use ``useMe()`` data passed down – ****never ``useCurrentUser()``****, which only works inside ``RequireAuth`` (a signed-in visit to ``/`` crashed because of this).
- Pages whose output depends on env vars (login/register show the Google button) are ``export const dynamic = 'force-dynamic'``. A prerendered page freezes env at build time, so adding keys on Vercel would need a rebuild.
- "Confirm password" is client-only: strip ``confirmPassword`` before sending.
- No ``setState`` inside ``useEffect`` to "reset" UI on navigation – derive it (the mobile menu stores the pathname it was opened on).
- No reading ``ref.current`` during render – use the "previous value in state" pattern.
- ``react-toastify`` v11 injects its CSS at runtime, after ours. Theme overrides are scoped to ``:Toastify``, not ``:root``, or toasts stay white in dark mode.
- Set ``color-scheme: dark`` in ``dark``, or native selects and date pickers render light.
- A filter applied from outside the filter panel (clicking a ``#tag``) must be visible and removable – that's why active-filter chips sit outside the panel.
- Visible Delete buttons need a confirmation step (in the card, not a browser ``confirm()``), or one mis-tap loses a task.
- Status tabs ("All 5") and category tabs ("All categories") both start with "All" – in tests match the status tab with a regex like ``^All \d``.
- The page-level "New task" button is mobile-only (``sm:hidden``); the navbar has it from ``sm`` up. Don't show two on desktop.
- The dashboard must show newly quick-added tasks (the "Up next" section), otherwise quick add looks broken for tasks without a due date.
- Profile photos use a plain ``<img>`` (lint rule disabled on that line): the URL is same-origin, session-protected and versioned with ``?v=``, so ``next/image`` would need ``images.localPatterns`` for the query string and adds nothing.
- ``axiosClient`` defaults to ``Content-Type: application/json``, which makes axios JSON-encode a ``FormData``. File uploads go through ``api.upload``, which sends the Blob as the raw body with its own type.
- ****A cache-side version counter is not enough.**** A counter kept in memcached can be lost or reset (eviction, flush) or be unreachable during a write, which brings stale entries back. The version lives on the user in MongoDB (``dataVersion``), which ``requireUser`` already loads.
- ****Per-call ``mutate(..., { onSuccess/onSettled })`` callbacks are dropped if the component that called ``mutate`` unmounts first.**** The mobile menu once closed itself before logout finished, so the "Logged out" toast and redirect never ran. Close/navigate *inside* the callback.
- A parent with ``backdrop-filter`` (the sticky header's ``backdrop-blur``) becomes the containing block for ``position: fixed`` children, so the mobile menu is a ****sibling**** of ``<header>``, not a child.
- ``useDismiss`` listens for ``pointerdown``, not ``click``: the click that opens something would otherwise be seen as "outside" once it mounts; and give the opener as ``ignoreRef``.
- Pages that read ``?token=`` need ``<Suspense>`` like any ``useSearchParams`` page, and are

- `robots: noindex` + `referrer: no-referrer`.
- In JSX text use typographic apostrophes (') – straight `` fails `react/no-unescaped-entities`.

****Server****

- Bloom filter hashes must stay unsigned (`>>> 0`) – see Section 4.6.
- `bcrypt` ignores bytes past 72 → password max is 72.
- Registering no longer returns a session or a user: `useRegister` must not touch the auth cache.
- Mongoose `.lean()` does ****not**** apply schema defaults, so a field added later (like `hasPassword`) is simply missing on old documents. Derive from the source of truth where you can, and query `null` (which matches missing).
- `.lean()` returns BSON `Binary` for Buffer paths – `avatar.controller.ts` converts it with `toBuffer`.
- `structuredClone` strips the prototype from Mongoose `ObjectId`s (String(id) breaks) – the fake model uses its own `clone`.
- Env vars are read lazily (`env()`), so `next build` works with no secrets.
- ****`route()` connects to MongoDB before running the handler.**** A handler that never touches the database must not use it: on a cold serverless start the connection takes seconds. `logout` is exported bare (`export const POST = logout`) for this reason; a test asserts it never calls `connectDB`.
- `SESSION\_SECRET` under 32 characters stops every login route. In development the 500 response carries a `devDetail` field (and the terminal logs it) saying exactly what's wrong – via `describeError` in `server/lib/http.ts`, which also turns MongoDB connection errors into a checklist. Never shown in production.
- The Google routes are ****not**** wrapped in `route()`: they're full-page browser navigations, so any failure redirects to `/login?error=google\_failed` instead of stranding the user on raw JSON, and the start step needs no database.
- MongoDB Atlas Network Access must allow `0.0.0.0/0` for Vercel.

10. Deployment (Vercel)

1. Push to GitHub.
2. In MongoDB Atlas → Network Access, allow `0.0.0.0/0`. Copy the connection string.
3. Import the repo in Vercel (framework: Next.js, no build settings to change).
4. Environment variables:
 - `MONGODB\_URI` – the Atlas connection string
 - `MONGODB\_DB` – `taskora`
 - `SESSION\_SECRET` – 32+ random characters (`openssl rand -base64 32`)
 - Optional, for Google sign-in: `GOOGLE\_CLIENT\_ID`, `GOOGLE\_CLIENT\_SECRET`, and `APP\_URL` (e.g. `https://taskora.vercel.app`, no trailing slash).
 - For sign-up emails and password reset: `BREVO\_API\_KEY` and `EMAIL\_FROM` (e.g. `Taskora <you@yourdomain.com>`). The sender must be verified in Brevo first (Senders, domains & dedicated IPs); see Section 4.5c for the Yahoo/Gmail caveat. Without both, sign-up answers 503.
 - Optional read cache: `MEMCACHIER\_SERVERS`, `MEMCACHIER\_USERNAME`, `MEMCACHIER\_PASSWORD` (from your MemCachier cache's Analytics/Settings page, same as Eventra). `/api/health` shows `"cache": "configured"`.
5. Deploy, then open `/api/health` → `"database": "connected"` and `"email": "configured"`.

****Set `APP\_URL` even without Google****: canonical links, the sitemap and share previews use it (Section 6b)

****Setting up Google sign-in (Google Cloud Console):****

1. APIs & Services → OAuth consent screen: External; app name Taskora; your email.
2. Credentials → Create credentials → OAuth client ID → Web application.
3. Authorized redirect URIs – add both, exactly:
 - `http://localhost:3000/api/v1/auth/google/callback` and
 - `https://<your-vercel-domain>/api/v1/auth/google/callback`
4. Copy the client ID and secret into `.env.local` and Vercel. While the consent screen is in "Testing", only the test users you list can sign in – publish it for everyone.
6. Register a real account on the live site and create a task to confirm.

Changing `SESSION\_SECRET` signs everyone out (old cookies can't be unsealed) – that's expected.

11. How to add a feature (recipe)

1. ****Spec it here first**** (Section 1): what the user sees, rules, limits.
2. Shared rules → `constants/todo-values.ts` (enums, `LIMITS`) and `lib/validation.ts` (zod).
3. Model change → `server/models/\*.model.ts` (+ index if you'll query by it). If it's tenant-owned, follow the checklist in Section 4.9.
4. Logic → `server/controllers/\*.controller.ts`; wire in `app/api/v1/\*\*/route.ts` with `route()`.
5. Tests → `server/test/api.test.ts` (+ extend `fake-model.ts` if needed).
6. Client → `types/`, `api/` fetcher, `hooks/` (query or optimistic mutation), component, view.

7. Run typecheck, lint, test and build; check mobile + desktop, light + dark.
8. Update this file: the feature (Section 1), the API table (Section 4.4), and any new gotcha (Section 9).

12. Milestones

Work top to bottom. Each milestone ends with typecheck, lint, test and build passing (Section 3), then a review and a commit (`feat: milestone N – <name>`). Update the status here when a milestone is finished.

Milestone 1: Project foundation

- Next.js 16 + TypeScript + Tailwind v4 + ESLint flat config + Vitest.
- Folder structure from Section 4.1; `constants/site.ts`, `constants/todo-values.ts`.
- Design tokens, fonts, dark/light theme with the pre-paint script (Section 6).
- `server/config/env.ts` (lazy) and `server/config/db.ts` (cached connection); `/api/health`.
- ****Done when:**** `npm run dev` shows a themed placeholder page, the theme toggle works without a flash, and `/api/health` reports the DB state.

Milestone 2: Accounts and sessions

- User model, shared validation (`lib/validation.ts`), `route()` wrapper and envelope (Section 4.3).
- Register, login, logout, me; sealed cookie sessions; dummy-hash login timing (Section 4.5).
- Username bloom filter + availability endpoint (Section 4.6); rate limits (Section 4.8).
- `proxy.ts`, `RequireAuth`, `GuestOnly`; login and register pages with live username check.
- ****Done when:**** API tests cover register/login/logout/me, duplicates, reserved usernames, forged cookies and rate limiting; a user can sign up, reload and stay signed in.

Milestone 3: Tasks API

- Todo model with notes, priority, category, tags, dueDate, subtasks, pinned (Section 1.3).
- List with every filter and sort, stats, get, create, update, subtask toggle, delete, clear completed (Section 4.4).
- ****Done when:**** API tests prove privacy (another user's task → 404), every filter and sort, stats, and validation errors.

Milestone 4: Tasks UI

- Navbar (signed-in/out, mobile menu, avatar menu); landing page.
- All tasks page: quick add, status tabs, category tabs, search, Filters panel, filters in the URL, active-filter chips, clear completed.
- Task cards in a grid with Edit / Delete buttons and in-card delete confirmation; compact rows for the dashboard.
- Ink strike-through, pin, subtasks inline, tag click to filter; optimistic updates (Section 5).
- Landing page hero illustration (SVG, token colours).
- New task and edit task pages sharing one form (tags input, subtasks editor, pin switch).
- ****Done when:**** in a browser, create a full task, complete it, filter by tag, category and due date, edit it, and refresh without losing the view.

Milestone 5: Dashboard and profile

- Dashboard: weekday headline, greeting, progress, stats, quick add, Overdue, Due today, Up next, by category.
- Profile: edit name/username/email/bio with live username check, change password (signs out other devices), delete account.
- ****Done when:**** API tests cover profile, password change and deletion; the full sign-up → tasks → profile → delete journey works at 400px and desktop, light and dark.

Milestone 6: Ship

- README, `.env.example`, security headers, 404 and error pages.
- Deploy to Vercel (Section 10); confirm `/api/health` says connected on the live URL.
- ****Done when:**** a real account can be created and used on the live site.

Milestone 7: Google sign-in and auth polish

- Continue / Sign up with Google (PKCE + state, verified email only, link by email, generated username via the bloom filter).
- Google-only accounts: set a password, delete by typing the username, "Google connected" badge.
- Confirm password on sign-up; login and register in a bordered card with the logo above it.
- Landing feature cards open a "Create a free account to continue" modal when signed out.
- ****Done when:**** Google tests pass with stubbed endpoints; `?error=` codes show friendly messages; no hydration errors in dev mode (dark and light); signed-in visits to `/` and a 404 page work.

Milestone 8: Undo and keyboard

- Undo toast (5s) after deleting a task and after "clear completed" (`POST /todos/[id]/restore` already exists – the trash).
- Keyboard shortcuts: `n` new task, `/` focus search, `x` toggle the focused task; a `?` help popover.
- ****Done when:**** undo restores exactly what was deleted (tests), shortcuts never fire while typing in an input.

☒ Milestone 9: Password reset and email verification

- "Forgot password" email with a single-use, 30-minute token (hashed in the DB); reset page.
- Email verification on sign-up – ****required before login**** (stricter than the original "unverified users can use the app with a banner", at the user's request); accounts from before it are grandfathered. Changing your email goes through a pending address.
- Email via Brevo (first written for Resend, switched because Resend needs a verified domain); `BREVO\_API\_KEY` / `EMAIL\_FROM` in the env docs.
- Also done alongside: special character required in new passwords; redesigned mobile menu; outside-press dismissal everywhere (`useDismiss`).
- ****Done when:**** tests cover token expiry, reuse and a wrong token; login still gives one generic error.

✓

☒ Milestone 10: Profile photos (built before 8 and 9, at the user's request)

- Upload (≤ 2 MB, jpg/png/webp), crop to square in the browser, initials as the fallback.
- Decision: stored in MongoDB on the user (the cropped image is ~20–60 KB) instead of Cloudinary, so there's no extra service or API key, and deleting the account deletes the photo. Move to Cloudinary or Vercel Blob only if photos get bigger or need a CDN.
- ****Done when:**** upload, replace and remove all work; the file type is checked server-side, not just in the browser.

☒ Extra: Trash (soft delete)

- Delete and "clear completed" move tasks to the trash; restore, delete forever, empty trash (Section 1.3).
- ****Done when:**** tests prove trashed tasks are hidden everywhere, restore is exact, permanent delete only works from the trash, and trash is private per user.

☐ Milestone 11: Recurring tasks and reminders

- Repeat daily / weekly / monthly; completing one creates the next occurrence.
- Optional due time + browser notification reminder.
- ****Done when:**** tests cover the next-occurrence dates, including month ends (31 Jan → 28/29 Feb).

☐ Milestone 12: Organise

- Drag-and-drop ordering inside the pinned and unpinned groups (persisted).
- Custom categories per user; tag autocomplete from existing tags.
- Export tasks as CSV / JSON.
- ****Done when:**** order survives reload and other devices; export opens cleanly in a spreadsheet.

☒ Extra: MemCachier read cache

- Per-user, encrypted, version-invalidated cache of the task list, stats and trash (Section 4.7b), in Eventra's format.
- ****Done when:**** tests prove a hit skips MongoDB, every write is visible on the next read, users never share entries, the cache holds only ciphertext, and a down cache changes nothing for the user. ✓

☐ Milestone 13: Hardening

- Shared rate limiting (replace the in-memory limiter). MemCachier is now available for this too (counters with `increment` + expiry) instead of Upstash, but `rateLimit()` is synchronous today and would become async.
- (Later) Shared workspaces/teams – changes the tenant from user to workspace; see Section 4.9.
- Playwright e2e suite in the repo, run in CI (GitHub Actions) on every PR.
- ****Done when:**** CI blocks a PR that fails typecheck, lint, tests, build or e2e.